Nginx API Gateway Uygulaması

Python HTTP Server + Nginx + Docker Compose | Adım adım Ubuntu rehberi

Bu çözümde Flask veya FastAPI kullanmadan, Python standart kütüphanesindeki http.server ile iki bağımsız HTTP servisi oluşturacağız. Nginx gelen istekleri URL yoluna göre ilgili servise aktaracak; bütün bileşenler Docker Compose ile başlatılacak.

1. Hedef ve mimari

http://localhost:80/login → HTML giriş sayfası
http://localhost:80/logout → Çıkış yapılmıştır.

```
Tarayıcı (localhost:80)
          |
          v
      Nginx :80
       /      \
  /login      /logout
     |           |
     v           v
login-api     logout-api
  :8000          :8001
    (Docker Compose bridge ağı)
```

Nginx bir reverse proxy görevi görür. Backend portları host makinesine yayımlanmaz; Nginx servis adlarıyla Docker ağı üzerinden erişir.

2. Klasörleri oluştur

```
mkdir nginx-http-server
cd nginx-http-server
mkdir login-api logout-api nginx
```

```
nginx-http-server/
├── login-api/
│   ├── app.py
│   └── Dockerfile
├── logout-api/
│   ├── app.py
│   └── Dockerfile
├── nginx/
│   └── default.conf
└── docker-compose.yml
```

Bu projede requirements.txt gerekmez: http.server Python ile birlikte gelir.

3. Login API: login-api/app.py

```
nano login-api/app.py
```

```
from http.server import BaseHTTPRequestHandler, HTTPServer

class LoginHandler(BaseHTTPRequestHandler):
    def do_GET(self):
```

```
        if self.path == "/login":
            html = """<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8">
  <title>Login</title>
</head>
<body>
  <h1>Giriş Sayfası</h1>
  <form>
    <input type="text" placeholder="Kullanıcı Adı">
    <br><br>
    <input type="password" placeholder="Şifre">
    <br><br>
    <button type="button">Giriş Yap</button>
  </form>
</body>
</html>"""
            self.send_response(200)
            self.send_header("Content-type", "text/html; charset=utf-8")
            self.end_headers()
            self.wfile.write(html.encode("utf-8"))
        else:
            self.send_error(404, "Sayfa bulunamadi")

server = HTTPServer(("0.0.0.0", 8000), LoginHandler)
print("Login API 8000 portunda calisiyor", flush=True)
server.serve_forever()
```

do_GET() GET isteğini işler; self.path istenen URL yoludur. send_response(200) HTTP durum kodunu, send_header() içerik türünü belirtir. wfile.write() yanıt gövdesini gönderir.

4. Logout API: logout-api/app.py

```
nano logout-api/app.py

from http.server import BaseHTTPRequestHandler, HTTPServer

class LogoutHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/logout":
            message = "Çıkış yapılmıştır."
            self.send_response(200)
            self.send_header("Content-type", "text/plain; charset=utf-8")
            self.end_headers()
            self.wfile.write(message.encode("utf-8"))
        else:
            self.send_error(404, "Sayfa bulunamadi")

server = HTTPServer(("0.0.0.0", 8001), LogoutHandler)
print("Logout API 8001 portunda calisiyor", flush=True)
server.serve_forever()
```

Logout servisi HTML yerine text/plain döndürür. Her iki servis de 0.0.0.0 adresinde dinlediği için diğer container'lar tarafından erişilebilir.

5. Dockerfile dosyaları

login-api/Dockerfile dosyasını oluştur:

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
EXPOSE 8000
CMD ["python", "app.py"]
```

logout-api/Dockerfile dosyasını oluştur:

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
EXPOSE 8001
CMD ["python", "app.py"]
```

FROM temel image'ı seçer; WORKDIR çalışma klasörünü belirler; COPY dosyayı image'a ekler; CMD sunucuyu başlatır. EXPOSE portu host'a açmaz.

6. Nginx: nginx/default.conf

```
nano nginx/default.conf

server {
    listen 80;
    server_name localhost;

    location = /login {
        proxy_pass http://login-api:8000;
    }

    location = /logout {
        proxy_pass http://logout-api:8001;
    }
}
```

location = /login yalnızca tam /login yolunu eşleştirir. proxy_pass isteği ilgili Python sunucusuna aktarır. Docker DNS, login-api ve logout-api servis adlarını çözümler.

7. Docker Compose: docker-compose.yml

```
nano docker-compose.yml

services:
  login-api:
    build: ./login-api
    expose:
      - "8000"
    networks:
      - api-network

  logout-api:
    build: ./logout-api
    expose:
      - "8001"
    networks:
      - api-network

  nginx:
    image: nginx:stable
    ports:
      - "80:80"
```

```
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - login-api
      - logout-api
    networks:
      - api-network

networks:
  api-network:
    driver: bridge
```

ports: "80:80" host portu 80'i Nginx'e bağlar. expose backend portlarını belgelemek içindir; host'a yayımlamaz. depends_on başlatma sırasını düzenler, servislerin tamamen hazır olduğunu garanti etmez.

8. Başlatma ve test

Docker Engine ve Docker Compose eklentisinin kurulu olduğundan emin ol:

```
docker --version
docker compose version
sudo systemctl status docker
```

Projenin ana klasöründe çalıştır:

```
docker compose config
docker compose up -d --build
docker compose ps
```

Tarayıcıdan http://localhost/login ve http://localhost/logout adreslerini aç. Terminalde HTTP durum kodunu ve içeriği test et:

```
curl -i http://localhost:80/login
curl -i http://localhost:80/logout
```

Her iki adreste de 200 OK beklenir. /login HTML sayfası; /logout ise "Çıkış yapılmıştır." metnini döndürmelidir.

9. Ağ üzerinde istek nasıl ilerler?

1) Tarayıcı localhost:80 adresine TCP bağlantısı kurar.
2) Docker port yönlendirmesi isteği Nginx container'ına ulaştırır.
3) Nginx URL yolunu inceler ve uygun location kuralını seçer.
4) Docker DNS backend servis adını container IP'sine çözümler.
5) Nginx backend'e ayrı HTTP isteği gönderir.
6) Python do_GET() çalışır, HTTP yanıtı üretir.
7) Nginx yanıtı tarayıcıya iletir.

10. Loglar, hata ayıklama ve durdurma

```
docker compose logs -f nginx
docker compose logs login-api
docker compose logs logout-api
docker compose down
```

502 Bad Gateway: Backend ayakta olmayabilir; logları kontrol et.
Port is already allocated: Host 80 portu başka işlem tarafından kullanılıyor olabilir. sudo ss -ltnp '(sport = :80)' ile kontrol et.
404 Not Found: URL yolunun tam olarak /login veya /logout olduğundan emin ol.

11. Ödev kontrol listesi

☐ İki bağımsız Python HTTP servisi hazır.
☐ Her servis için Dockerfile oluşturuldu.
☐ Nginx URL yoluna göre doğru servise yönlendiriyor.
☐ Docker Compose üç container'ı başlatıyor.
☐ /login HTML döndürüyor.
☐ /logout istenen metni döndürüyor.

Not: Bu giriş formu yalnızca görsel amaçlıdır; gerçek kimlik doğrulama yapmaz. Python http.server üretim ortamı için önerilmez; bu çözüm eğitim ve ödev amacıyla hazırlanmıştır.